



Ink Morrow

WHERE STORIES GROW CLAWS

5.0 EDITION

Maintainer, Testing & Release Handbook

Change playable fiction without breaking history or trust

For contributors and release owners / September 2026

Maintainer, Testing & Release Handbook

Change the playable-fiction product without losing causal history, paid authority or honest evidence. This handbook describes current development and release practice, not the retired writing-suite workflow.

Contents

Repository and product ownership

Plan a substantial change

Local verification layers

Data and concurrency qualification

UI, accessibility and one logo

The six-book documentation workflow

CI, release and exact-head merge

Definition of done and regression response

Optional quality verification

People and episode verification

Fourth-wall verification

Clear-influence verification

Fair-resistance verification

Illustrated-save verification

01 Repository and product ownership

Read AGENTS.md and the current release decisions before changing behaviour. The approved 5.0 product defaults to a reader-director outside the cast, excludes manual prose authoring and does not promise 4.x data migration.

The authoritative project checkout is under WSL at /home/rthorman/src/ink-morrow-5; shared Git metadata lives under /home/rthorman/src/ink-morrow/.git. Windows tools may work through WSL paths. Do not create a second source copy or recover abandoned older branches.

backend/src/app.js is the composer. Current game code lives in modules/fiction; the frontend starts its fiction app, not the old room router. Auth, provider, media and publication infrastructure is reused. legacy-runtime.js is opt-in for inherited tests, never enabled by server.js.

Treat existing user changes as owned work. Preserve unrelated edits, use a coherent feature branch and inspect the diff before committing. A failed command or CI rerun does not authorise history rewriting, deployment or data cleanup.

Project-owned 5.0 material remains AGPL-3.0-only. Preserve third-party licenses, bundled-font notices and the historical development credits. Do not describe generated fixtures as human playtesting or an unrun benchmark as research evidence.

02 Plan a substantial change

Choose a coherent user outcome and identify its state, UI, privacy, cost and documentation boundaries before implementation. Batches should be large enough to justify a full push/PR/CI/merge cycle, without combining unrelated risks.

For each mutation, name the input authority, expected revision, transaction, failure outcome and immutable record. For each provider call, name the role, data sent, maximum calls, consent scope, timeout, stale checks and known/unknown billing. For each read, name which path and visibility it can expose.

Retain existing deterministic invariants unless the owner explicitly changes them. New game design can challenge branding or compatibility, but that does not waive security, credential privacy, paid consent or data integrity.

Tests should accompany the behavioural change, not merely assert the new DOM. Include invalid input, delayed responses, navigation/lock, concurrent revision, restart and zero-provider paths where relevant. Use fixtures with exact expected effects rather than accepting whichever prose a model returns.

Update all affected manual sources and regenerate the full six-book set together. A substantial feature is incomplete if the user guide still describes retired controls or if a version label masks inherited operating instructions.

03 Local verification layers

From the repository root, run `npm run lint`, `npm run test:setup`, `npm run check:release` and `npm run check:brand`. Backend Jest runs through `node node_modules/jest/bin/jest.js` in backend. Frontend Jest uses `node --experimental-vm-modules node_modules/jest/bin/jest.js` in frontend.

Backend tests use isolated databases and injected providers. New production boundary tests instantiate the default composer without `legacyEnabled`. Inherited component tests may explicitly opt into the old runtime; their coverage is regression protection, not a claim of a live 5.0 feature.

Browser tests use port 3100, an in-memory database, fresh media and an isolated setup credential. Never point them at the owner's port 3000 or real database. Sweep abandoned E2E servers using the supplied script; do not kill processes by a broad command-line substring.

Run desktop Chromium and Mobile Chrome in separate invocations so each receives a fresh server. Use an existing browser-capable environment; do not download a browser merely to satisfy local verification. CI has its own browser setup.

A passing test count is evidence about that exact code/configuration. Record full-suite versus added focused cases accurately. Do not imply a paid provider, different browser engine or human session was tested when only mocks ran.

04 Data and concurrency qualification

The new family is ink-morrow-5, schema 22. Fresh creation, supported migration, repeat startup, rollback, ledger integrity, future-family refusal and source-file preservation need independent tests. Never edit a migration checksum merely to make a valuable database open.

Exercise old database files with WAL and missing shared-memory companions. Preflight must inspect a private copy, preserve original bytes/timestamps and not create source sidecars. Test orphan journals, symlinks, failed copy/storage and source changes. A current valid WAL-only committed state must recover.

For paid work, delay the provider, mutate or restart, then return the response. Assert no stale canon, complete known/unknown billing and no duplicate purchase. Quality requires per-call records and one shared repair allowance. Transport retry is disabled even on uncertain failure.

For graph changes, test exact fork snapshots, sibling exclusion, old fact retrieval beyond the working set, corrected/retired versions and evidence remapping through save import. Test all references before any import write.

Use long deterministic histories to test bounded readers and retrieval. A large fixture proves structural scale and invariants, not that an actual model will remember every relevant detail or remain enjoyable across a long campaign.

05 UI, accessibility and one logo

Immediate feedback is part of correctness. Dialogs should open before slow supporting requests; buttons must show reviewing/submitting status and prevent duplicate work. Cancelling, failing or changing routes should preserve or discard drafts according to the explicit lifecycle, never by accident.

Exercise keyboard navigation, focus restoration, labelled controls, mobile reflow, delayed loads and lock while a request is pending. Automated accessibility scans supplement real inspection; they do not prove a design is easy to use.

The canonical product lockup is `frontend/brand/ink-morrow-lockup.svg`, the existing artwork at the top of GitHub README. App header/footer, auth threshold and all manual covers/running headers reuse it. Do not regenerate it or emulate its lettering with another font. Ordinary product-name mentions remain text.

`npm run check:brand` includes a canonical-logo reference guard. Browser tests verify the loaded image in the reader and locked threshold. Inspect both narrow and wide screenshots for size and clipping; inspect rendered PDF covers and headers, not only SVG paths in source.

Images above manuscript text and separate preceding EPUB image pages are distinct requirements. Check every EPUB spine item and reader layout rather than assuming shared HTML produces both representations correctly.

06 The six-book documentation workflow

Active books are declared in `docs/pdf-library/books.mjs` and use Markdown sources with a shared `renderer/theme`. The previous fixed HTML guide remains a historical content and visual baseline, not a current workflow.

Update the User Guide, Operations, Architecture, State Machine Atlas, Security and Maintainer books where affected, then run `npm run docs:pdf`. All six PDFs and `generated.json` form one artifact set. The active filenames, cover labels, metadata and headers use 5.0.

Run strict freshness after rendering. PDF QA checks extractable text, replacement characters, current identity, outlines and page counts. Rasterize pages and inspect covers, tables, code, headers and final pages. A valid PDF hash cannot prove layout or absence of a clipped paragraph.

The guide includes complete journeys using no optional provider features, the whole current feature set and coherent subsets. No-provider play means existing reading/local management, not manual authoring or an invented offline narrator. Every journey identifies costs, skipped features and its save/book outcome.

Retain historical release evidence, but make current entry-point documentation unambiguous. Never claim a book is current merely because its filename changed. A changed source or brand asset requires regenerating the complete set.

07 CI, release and exact-head merge

The five CI gates are Brand residue, Lint, Jest, Production dependency audit and Playwright E2E. The brand job also checks release identity and warns about PDF freshness. For release work, run strict freshness locally even though the general CI freshness check is advisory.

Feature PRs target release/5.0.0. Push a coherent batch, inspect the PR diff and wait for all five gates on its exact current head. A green earlier commit is not authority to merge a changed head. Use a head-matching merge and verify the resulting merge commit.

After each feature merge, update from the integration head before the next batch. Do not rewrite the historical 4.x release line. Unrelated dependency PRs are not part of this product programme merely because they are open.

Final release-to-main integration is authorised only after all approved 5.0 work is complete and the final PR's current head is green. Record actual PR links, head/merge hashes, tests and remaining limitations in the release record. Do not label planned checks as passed.

Merging code is not deployment. The owner requested the old port-3000 instance stopped; do not restart or replace it without a separate request. A final handoff should briefly distinguish code state, running state and 4.x compatibility.

08 Definition of done and regression response

A completed change has a verified user path, bounded state transition, correct privacy/cost boundary, failure and race coverage, current documentation and an honest release record. For model-facing changes, distinguish protocol tests from semantic evaluation and do not promise unmeasured quality gains.

A completed 5.0 release additionally has fresh-family storage isolation, no live manual-authoring or retired room APIs, consistent branding, complete generated manuals and green exact-head integration. Old user data remains untouched. Character/template portability is not a release blocker and is not advertised unless implemented.

When a regression appears, reproduce it with minimal fixture data. Determine whether the defect is product code, a stale test assumption, environment, provider behaviour or external infrastructure. Do not weaken an invariant simply to restore a green badge.

Fix a proven defect in a focused but substantial batch, rerun the affected layer and full release gates in proportion to risk, and record the evidence. An intermittent registry error may justify a CI rerun; it does not justify bypassing the audit.

If completion needs a new product choice, paid live evaluation, deployment authority or destructive recovery, pause and ask. Persistence toward a merge does not broaden the authorised actions. Keep the owner informed with concise, meaningful updates rather than unchanged polling reports.

09 Optional quality verification

Fixture tests cover Off/Standard/Memory/Both, first-pass acceptance, the sole structural or consistency repair, replacement re-review and terminal rejection. Assert maximum totals of one/four/six, role-specific model routing and disabled transport retries. Reject malformed or unevidenced review JSON; approval must not bypass deterministic ownership, fourth-wall, state-effect or adjudication checks. No canon mutation may happen between draft and final acceptance.

Test missing/stale plan identity, unavailable memory before the first purchase, provider changes after an already billed draft, mixed known/unknown costs, restart and late results, zero-call unchanged rulings and free idempotency replay. Rewind/save tests preserve quality choices and aggregate spend without carrying request authority. UI tests require fresh scoped review despite old global consent, cancelled-direction retention, free progress reads and lock/navigation fencing. Exercise the real preference API and both desktop/mobile browser journeys.

These automated protocol fixtures make no paid provider requests and establish no model-quality ranking. Live comparisons need separate authorisation and versioned results: model/provider/date, repeated-persuasion and justified-cooperation cases, knowledge/ownership/continuity errors, review false positives, latency and actual known/unknown cost. Never claim the extra calls guarantee better play.

10 People and episode verification

Exercise both authored openings from development to payoff and aftermath, with Follow/Steer sufficient and no compulsory avatar. The garden fixture covers a refused group role, a free unchanged repeat and cooperation on new agreed terms. The mystery fixture distinguishes affection from restored practical trust. These are protocol fixtures, not evidence from human playtesting or paid models.

Test relationship aspects/targets, exact evidence, owned-character boundaries, rejection of numeric meters and world-fact rewriting, historical provenance, reload/fork/save restoration and remapped payoff identities. Test quiet scenes, clarification and local correction cannot manufacture a played payoff. Ending early remains possible; goal completion never forces an episode end.

Recaps must find narrated moments despite many local changes, filter public commitments/relationships before limits, exclude retired and other-path records, and make zero provider calls. Browser tests cover immediate recap feedback, episode questions, reload, narrow reflow and retained path/episode drafts after failed writes. Include optional standard/memory/both consistency and fresh-storage isolation in the same final integration regression evidence.

11 Fourth-wall verification

Test default Never, invalid choices, Living-world-only UI visibility, local preference saves and restored settings. Cover optional consecutive Freely addresses, the five intervening narrated scenes required by Rarely, and no cooldown advancement on failure or clarification. Reject unknown/inhabited speakers and effects evidenced only by an aside. Verify known charges survive rejection and, with quality Off, no automatic extra request occurs.

Reload, fork and save-copy must preserve branch-local settings and counters; reject future scene indices in imported saves. Addresses remain ordinary saved passage text for publication. Mocked protocol tests do not demonstrate semantic model compliance with Never or prove resistance quality. The optional consistency pipeline has its own verification contract above; fourth-wall permission alone does not enable it or authorise another model call.

12 Clear-influence verification

Cover the default one-moment direction, explicit ongoing focus, failure retention, local focus release, rewind and save-copy restoration. Invitations must not submit, overwrite drafts, reveal hidden facts or supply an inhabited person's decisions. Challenge preflight and repeat must make zero provider calls; stale preflight must not bypass paid consent. Test navigation during review as well as generation.

Exercise recall beyond 128 working facts, public filtering before the 32-result limit, retirement and sibling-path rejection. Changed-fact citations must point to real earlier records, and portable copies must remap them. Exercise shelf paging beyond 200 stories, immediate loading feedback, keyboard operation and 320-pixel reflow. Automated accessibility scans supplement, not replace, human usability testing. All six books remain one regenerated artifact set per feature PR.

13 Fair-resistance verification

The deterministic royal-guard fixtures test twenty repeated/paraphrased requests without extra purchases, legitimate new recorded authority, unknown or hidden evidence, distinct styles, narrator outcome disagreement, private-field filtering, branch restoration and save import. Memory fixtures exceed 128 facts, retrieve an old relevant fact, correct and retire it, and exclude a sibling path's history. Run `node node_modules/jest/bin/jest.js --runInBand tests/fiction-resistance.test.js` from backend. These use mocked providers and do not establish model suitability.

Live evaluations need an explicitly approved paid budget and repeatable fixtures. Record exact model/settings/date, latency, cost, contradictions, unjustified capitulation AND unjustified stubbornness. Human semantic review is necessary; another model's approval is not sufficient. No model is certified by unit tests. The owner approved the three substantial iteration PRs and final green integration to main. Deployment remains separate; the old port-3000 instance is now stopped.

14 Illustrated-save verification

Keep the media/save batch substantial. Test synchronous dialog feedback, prevented duplicate image purchases, retained drafts on failure, late-response route/lock isolation, exact path-local placements, safe upload and local description correction. Test EPUB image-only spine resources before associated prose, while manuscript/HTML places the image above that prose. Re-read the whole EPUB spine, not only book.xhtml.

Save tests must round-trip hidden truth, knowledge, commitments, resources, control, episodes, preferences, director history, all branches, illustrations and known/unknown spend. Reject malformed versions/fields, cycles, dangling/future/cross-path references, unsafe media and corrupt digests before writes. Inject import transaction failure and verify both database rollback and cleanup of only newly staged image files. Browser journeys must upload, export, download a private save and import a copy. Never use real provider spending to satisfy deterministic tests. Verify final 5.0 identity, fresh storage and the complete current documentation together.

Visual catalogue coverage must exercise world, character and Scribe CRUD and both upload/generation paths, plus story covers and all reference portraits. Verify auth/CSRF before uploads, one pending purchase, changed-provider rejection, failed and late known/unknown charges, safe normalization and owned-file cleanup. Ordinary details, selection and upload make no model request. Old authoring routes stay 404.

Reference-development coverage must exercise all three typed output shapes, empty and partial seeds, private-field disclosure, exact provider/model review, idempotent replay, one invalid-output correction, combined known/unknown spend and restart interruption. Browser coverage proves the AI result fills an editable unsaved studio and that Save & add image moves to the explicit upload/painting boundary without a hidden purchase.

Freeze and copy selected images at setup. Delete or change the catalogue, then check that the story and its images remain unchanged. Round-trip private references and all visual asset IDs through saves; reject dangling or malformed copies. Books must contain the cover and passage art, never private lore or reference portraits. Test schema-22 to 23 migration with identical earlier ledger rows. UI tests cover immediate dialogs, retained selection/drafts, bounded tablet images, catalogue pagination, lock clearing and stale responses. Use mocked model output, not live spend.